

Letter Combinations Demonstration

Title: Letter Combinations of a Phone Number using Java Collection Framework

Objective / Aim :

To write and execute a Java program to find all possible letter combinations represented by a string of digits from 2 to 9 using telephone keypad mapping and the Collection Framework.

System Requirements :

Hardware:

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage
- Keyboard and Monitor

Software:

- Operating System: Windows
- Programming Language: Java
- Java Development Kit (JDK 17 or JDK 21)
- Visual Studio Code / Eclipse IDE or any Java IDE
- Command Prompt/Terminal

Algorithm / Logic Description :

1. Start the program.
2. Initialize a telephone keypad mapping array for digits 2 to 9.
3. Read the input string of digits from the user.
4. Create an empty ArrayList to store the combinations.
5. If the input is empty, display an empty list.
6. Check whether all input digits are between 2 and 9.
7. For each digit, obtain its corresponding letters from the keypad mapping.
8. Use recursion to combine each letter with the letters of the remaining digits.
9. Add each complete combination to the result list.
10. Display all possible combinations.
11. Stop the program.

UML Class Diagram :

LetterCombinations

- phone : String[]
- result : List<String>
- + main(args : String[]) : void {static}
- + generate(digits, index, current) : void

|

∨

ArrayList<String>

- + add(element) : boolean
- + toString() : String

Relationship: The LetterCombinations class utilizes the ArrayList class from the Java Collection Framework to store and output generated combination sequences.

Source Code :

File: LetterCombinations.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
```

```
public class LetterCombinations {

    static String[] phone = {
        "", "", "abc", "def", "ghi", "jkl",
        "mno", "pqrs", "tuv", "wxyz"
    };

    static List<String> result = new ArrayList<>();

    static void generate(String digits, int index, String current) {

        // Store the combination when all digits are
        // processed if (index == digits.length()) {
        result.add(current);
        return;
        }

        int digit = digits.charAt(index) - '0';
        String letters = phone[digit];

        // Try every letter mapped to the current digit
        for (int i = 0; i < letters.length(); i++) {
            generate(digits, index + 1,
                current + letters.charAt(i));
        }
    }
}
```

```

public static void main(String[] args) {
    Scanner sc = new
    Scanner(System.in);

    System.out.print("Enter digits: ");
    String digits = sc.nextLine();

    if (digits.isEmpty()) {
        System.out.println("Output:
        []");
    } else {
        boolean valid = true;

        for (int i = 0; i < digits.length(); i++) { if
            (digits.charAt(i) < '2' ||
            digits.charAt(i) > '9') {
                valid = false;
                break;
            }
        }

        if (valid) {
            result.clear();
            generate(digits, 0, "");
            System.out.println("Output: " + result);
        } else {
            System.out.println(
                "Enter digits from 2 to 9 only."
            );
        }
    }

    sc.close();
}
}

```

Compilation & Execution Commands :

Save file as: LetterCombinations.java

Compilation: javac LetterCombinations.java

Execution: java LetterCombinations

Output Screenshot :

```
PS C:\Users\anits\Desktop> javac LetterCombinations.java
PS C:\Users\anits\Desktop> java LetterCombinations
Enter digits: 23
Output: [ad, ae, af, bd, be, bf, cd, ce, cf]
PS C:\Users\anits\Desktop> █
```

Result :

Thus, the Java program to find all possible letter combinations of a phone number was implemented and executed successfully using the Java Collection Framework, including the List interface and ArrayList class.